Một số vấn đề JavaScript

JavaScript • Ví dụ tự chạy • Kết quả mong đợi

TS.GVC. Nguyễn Văn Khanh • PLC & IIoT Lab

1. Mục tiêu & cách học

Mỗi khái niệm đều có ví dụ ngắn để sinh viên chạy và đối chiếu kết quả.

- Đọc được JavaScript thường gặp khi gọi API, xử lý JSON, timeout và lỗi mạng.
- Biết chọn const, let; hạn chế var trong code JavaScript hiện đại.
- Hiểu callback, Promise, async/await — nền tảng quan trọng của lập trình mạng.
- Biết tự kiểm tra kết quả bằng Node.js hoặc Console của trình duyệt.

```
Cách tự kiểm tra nhanh:
1) Mở Chrome DevTools > Console, hoặc
2) Lưu file .js và chạy: node ten_file.js
```

2. var, let và const

Chọn đúng cách khai báo biến giúp code mạng ổn định, ít lỗi phạm vi biến.

const

Dùng cho giá trị không gán lại: URL API, port, cấu hình.

let

Dùng cho biến có thể thay đổi: số lần thử lại, trạng thái kết nối.

var

Không khuyến nghị trong JS hiện đại vì phạm vi hàm dễ gây nhầm lẫn.

```
const API_URL = 'http://localhost:3000/devices';
let retryCount = 0;

retryCount++;
console.log(API_URL, retryCount);
```

TDZ: với let/const, không sử dụng biến trước dòng khai báo. Lỗi xuất hiện sớm nên dễ sửa hơn.

3. TDZ là gì?

Temporal Dead Zone = vùng không được truy cập biến trước khi gặp dòng khai báo let/const.

Hiểu đơn giản

Biến đã thuộc phạm vi của block { ... }, nhưng chưa được phép dùng cho đến khi chạy tới dòng khai báo.

Khi nào gặp?

Gặp với let và const. Nếu truy cập sớm sẽ phát sinh ReferenceError.

Code gây lỗi

```
{
  console.log(status); // ReferenceError
  let status = 'online';
}
```

Code đúng

```
{
  let status = 'online';
  console.log(status); // online
}
```

Ghi nhớ: khai báo let/const trước khi sử dụng, đặc biệt trong các block if, for, try/catch.

Tự kiểm tra 1: biến và phạm vi

Chạy đoạn code và giải thích vì sao kết quả như vậy.

Code

```
let status = 'offline';

if (true) {
  let status = 'online';
  const port = 3000;
  console.log('inside:', status, port);
}

console.log('outside:', status);
```

Kết quả mong đợi

```
inside: online 3000
outside: offline
```

Câu hỏi: Nếu đổi let status trong block thành var status thì kết quả ngoài block thay đổi thế nào?

4. Optional chaining (?.)

Dùng để đọc dữ liệu JSON an toàn khi response từ server có thể thiếu trường.

Vấn đề

API không phải lúc nào cũng trả về đủ user, device, sensor, address...

Cách làm

Dùng ?. để đọc thuộc tính hoặc gọi hàm mà không làm chương trình bị lỗi.

Lưu ý

Nếu thiếu trường, kết quả là undefined. Không dùng ?. để gán giá trị.

```
const packet = {
  device: { id: 'D01', sensor: { temp: 31 } }
};

console.log(packet.device?.sensor?.temp);
console.log(packet.device?.ip);
```

```
31
undefined
```

Ứng dụng: đọc dữ liệu từ HTTP API, WebSocket hoặc MQTT message mà không cần nhiều tầng if.

Tự kiểm tra 2: đọc dữ liệu API thiếu trường

Hoàn thành dòng còn thiếu để không phát sinh lỗi.

```
const responses = [
  { device: { name: 'Pump', sensor: { temp: 29 } } },
  { device: { name: 'Valve' } },
  {}
];

responses.forEach(item => {
  const name = item.device?.name ?? 'unknown';
  const temp = item.device?.sensor?.temp ?? 'no data';
  console.log(name, '-', temp);
});
```

```
Pump - 29
Valve - no data
unknown - no data
```

?? là nullish coalescing: nếu vế trái là null hoặc undefined thì dùng giá trị dự phòng bên phải.

5. Hàm ẩn danh, callback và arrow function

Rất thường gặp khi xử lý sự kiện mạng và dữ liệu trả về từ server.

Hàm ẩn danh

Hàm không có tên; thường gán vào biến hoặc truyền trực tiếp làm tham số.

Callback

Hàm được truyền vào hàm khác và được gọi khi có sự kiện hoặc kết quả.

Arrow function

Cú pháp ngắn: (data) => { ... }. Phổ biến khi viết callback.

```
// Hàm ẩn danh gán vào biến
const logData = function(data) {
  console.log(data);
};

// Arrow function dùng làm callback
[1, 2, 3].forEach(x => console.log(x));
```

Tự kiểm tra 3: thứ tự chạy của callback

Dự đoán thứ tự in ra trước khi xem kết quả.

```
function fakeRequest(callback) {
  console.log('A. start request');
  setTimeout(() => {
    callback('B. server response');
  }, 1000);
}

fakeRequest((data) => console.log(data));
console.log('C. code continues');
```

```
A. start request
C. code continues
B. server response
```

Giải thích: setTimeout mô phỏng thời gian chờ server. Trong lúc chờ, chương trình vẫn chạy dòng tiếp theo.

6. Toán tử ...: rest và spread

Cùng một ký hiệu nhưng ý nghĩa phụ thuộc vào vị trí sử dụng.

Rest parameter

Trong định nghĩa hàm: gom nhiều đối số thành một mảng.

Spread

Khi tạo mảng/object hoặc gọi hàm: trải phần tử ra.

```
function sendToMany(message, ...clients) {
  clients.forEach(c => console.log(message, '->', c
}

sendToMany('ping', 'clientA', 'clientB');
```

```
const base = { protocol: 'http', port: 3000 };
const server = { ...base, host: 'localhost' };
console.log(server);
```

Tự kiểm tra 4: gom client và ghép cấu hình

Chạy code, sau đó sửa thêm clientC để kiểm tra hàm có cần đổi không.

```
function broadcast(...clients) {
  return clients.map(c => 'send to ' + c);
}

const defaultConfig = { timeout: 3000, retry: 1 };
const config = { ...defaultConfig, retry: 3 };

console.log(broadcast('A', 'B'));
console.log(config);
```

```
[ 'send to A', 'send to B' ]
{ timeout: 3000, retry: 3 }
```

Câu hỏi: Vì sao retry là 3 chứ không phải 1? Gợi ý: thuộc tính phía sau ghi đè thuộc tính phía trước.

7. Bất đồng bộ trong lập trình mạng

Khi chờ mạng, file hoặc CSDL, JavaScript không nên làm đứng chương trình.

- Gửi request HTTP có thể mất thời gian vì phụ thuộc mạng và server.
- Đọc file cấu hình, truy vấn CSDL hoặc nhận dữ liệu socket cũng cần chờ.
- Trong lúc chờ, chương trình vẫn có thể xử lý tác vụ khác.
- Ba cách thường gặp: callback → Promise → async/await.

```
console.log('1. gửi request');

// Giả sử fetch cần chờ server
fetch('/api/devices');

console.log('2. code vẫn chạy tiếp');
```

Tư duy cần nhớ: “đợi kết quả” không có nghĩa là làm đứng toàn bộ chương trình.

8. setTimeout và setInterval

Hai công cụ thời gian thường dùng để timeout, thử lại hoặc polling server.

setTimeout

Gọi callback một lần sau một khoảng thời gian ms.

```
setTimeout(() => {
  console.log('Reconnect server');
}, 3000);
```

setInterval

Gọi callback lặp lại sau mỗi khoảng thời gian ms cho đến khi clearInterval.

```
const timerId = setInterval(() => {
  console.log('Check server');
}, 5000);

// clearInterval(timerId);
```

Trong thực tế, tránh polling quá dày vì có thể làm tăng tải server.

Tự kiểm tra 5: thứ tự thực thi timer

Dự đoán thứ tự log, sau đó chạy để kiểm tra.

```
console.log('A');

setTimeout(() => {
  console.log('B - after 0 ms');
}, 0);

console.log('C');
```

```
A
C
B - after 0 ms
```

Dù thời gian là 0 ms, callback vẫn được đưa vào hàng đợi. JavaScript chạy xong code hiện tại trước.

9. Promise: kết quả trong tương lai

Promise giúp xử lý thành công/lỗi rõ ràng hơn callback lồng nhau.

pending
Đang chờ kết quả.

fulfilled
Hoàn thành thành công và có dữ liệu.

rejected
Thất bại và có lỗi.

```
fetch('/api/devices')
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error(error));
```

Trong lập trình mạng, Promise xuất hiện rất nhiều khi gọi API, đọc file, truy vấn CSDL hoặc thao tác socket bất đồng bộ.

Tự kiểm tra 6: Promise thành công và thất bại

Thay true thành false để xem nhánh catch hoạt động.

```
function requestDevice(ok) {
  return new Promise((resolve, reject) => {
    if (ok) resolve({ id: 'D01', status: 'online' });
    else reject(new Error('Server error'));
  });
}

requestDevice(true)
  .then(device => console.log(device.status))
  .catch(err => console.log(err.message));
```

```
online

Nếu gọi requestDevice(false):
Server error
```

Quy tắc: resolve đi vào then(), reject hoặc throw đi vào catch().

10. Promise.all: chạy nhiều tác vụ song song

Phù hợp khi cần gọi nhiều API độc lập rồi chờ tất cả hoàn tất.

Khi dùng

Các request không phụ thuộc nhau: /devices, /users, /logs.

Cần nhớ

Chỉ cần một Promise reject thì Promise.all sẽ reject.

```
const p1 = Promise.resolve('devices');
const p2 = Promise.resolve('users');
const p3 = Promise.resolve('logs');

Promise.all([p1, p2, p3])
  .then(values => console.log(values))
  .catch(err => console.log(err.message));
```

```
[ 'devices', 'users', 'logs' ]
```

Tự kiểm tra: đổi p2 thành Promise.reject(new Error("fail")) rồi chạy lại.

11. async và await

Cách viết phổ biến nhất hiện nay để xử lý bất đồng bộ trong JavaScript.

async

Đặt trước function. Hàm async luôn trả về Promise.

await

Chỉ dùng bên trong async. Chờ Promise có kết quả rồi mới chạy tiếp trong hàm.

try/catch

Bắt lỗi mạng, lỗi HTTP hoặc lỗi xử lý JSON.

```
async function getDevice(id) {
  try {
    const res = await fetch('/api/devices/' + id);
    if (!res.ok) throw new Error('HTTP ' + res.status);
    const device = await res.json();
    console.log(device.name);
  } catch (err) {
    console.error('Error:', err.message);
  }
}
```

Tự kiểm tra 7: async/await không cần server thật

Dùng Promise.resolve để mô phỏng dữ liệu trả về từ API.

```
function mockFetchDevice() {
  return Promise.resolve({
    json: () => Promise.resolve({ name: 'Pump 01' })
  });
}

async function run() {
  const res = await mockFetchDevice();
  const data = await res.json();
  console.log(data.name);
}

run();
```

```
Pump 01
```

Câu hỏi: await thứ nhất chờ cái gì? await thứ hai chờ cái gì?

Gợi ý: nhiều API trả về response trước, sau đó mới chuyển body sang JSON.

12. Từ callback hell đến async/await

Mục tiêu là làm luồng xử lý mạng dễ đọc như các bước thuật toán.

```
// Callback lồng nhau: khó đọc khi thêm nhiều bước
connectServer((err, conn) => {
  if (err) return handleError(err);
  conn.getToken((err, token) => {
    if (err) return handleError(err);
    getDevices(token, (err, devices) => {
      if (err) return handleError(err);
      console.log(devices);
    });
  });
});
```

```
// async/await: các bước rõ ràng
async function loadDevices() {
  try {
    const conn = await connectServer();
    const token = await conn.getToken();
    const devices = await getDevices(token);
    console.log(devices);
  } catch (err) {
    handleError(err);
  }
}
```

13. Mẫu xử lý API nên nhớ

Kết hợp async/await, try/catch và optional chaining để xử lý dữ liệu mạng an toàn.

```
async function loadTemperature() {
  try {
    const res = await fetch('/api/device/D01');
    if (!res.ok) throw new Error('HTTP ' + res.status);

    const data = await res.json();
    const temp = data.device?.sensor?.temp ?? 'no data';
    console.log('Temperature:', temp);
  } catch (err) {
    console.error('Cannot load data:', err.message);
  }
}
```

Điểm kiểm tra

1. Có await khi gọi API
2. Có kiểm tra res.ok
3. Có try/catch
4. Có ?. và ?? khi đọc JSON

Bài tập: thay /api/device/D01 bằng API thật hoặc mockFetch, rồi đối chiếu kết quả.

14. Bài tập tự luyện cuối buổi

Sinh viên có thể làm ngay bằng Node.js hoặc Console trình duyệt.

- Viết hàm nhận mảng packet và in tên thiết bị + nhiệt độ, kể cả khi thiếu sensor.
- Viết hàm retryConnect dùng setTimeout để thử kết nối lại sau 2 giây.
- Dùng Promise.all để gọi song song 3 tác vụ giả lập: devices, users, logs.
- Viết lại một đoạn callback lồng nhau sang async/await và thêm try/catch.

```
Checklist nộp bài:
[ ] Code chạy không lỗi
[ ] Có kết quả console rõ ràng
[ ] Có xử lý trường hợp thiếu dữ liệu hoặc lỗi
```

15. Tóm tắt nhanh

Nắm chắc các ý này trước khi học Node.js, HTTP server hoặc WebSocket.

- Ưu tiên const; dùng let khi cần gán lại; tránh var trong code mới.
- Dùng ?. và ?? để đọc dữ liệu API an toàn.
- Callback hữu ích cho sự kiện nhưng dễ gây callback hell khi lồng nhiều tầng.
- Promise biểu diễn kết quả tương lai: pending, fulfilled, rejected.
- async/await giúp code mạng dễ đọc; try/catch giúp xử lý lỗi rõ ràng.

```
Mẫu tư duy:
Gửi yêu cầu → Chờ kết quả → Kiểm tra lỗi → Đọc JSON → Xử lý dữ liệu
```

16. Callback: ý tưởng cốt lõi

Callback là hàm được truyền vào hàm khác để gọi lại khi có kết quả hoặc sự kiện.

Cần hiểu theo 3 ý

1) Hàm A nhận một hàm B làm tham số.

2) Khi tác vụ hoàn tất, A gọi B để trả kết quả.

3) Trong lập trình mạng, callback thường chạy sau khi có dữ liệu hoặc sau khi xảy ra lỗi.

Mẫu callback hay dùng trong Node.js

callback(error, data)

Nếu error khác null → xử lý lỗi.

Nếu error là null → xử lý data.

Ví dụ: hàm nhận callback

```
function readSensor(callback) {
  setTimeout(() => {
    callback(null, { id: 'D01', temp: 30 });
  }, 1000);
}

readSensor((err, data) => {
  if (err) return console.log('Error:', err);
  console.log(data.id, data.temp);
});
```

Kết quả mong đợi

```
D01 30
```

17. Callback: kiểm tra thứ tự chạy

Sinh viên nên dự đoán kết quả trước khi chạy code.

Code tự kiểm tra

```
console.log('1. bắt đầu');

function getDevice(callback) {
  console.log('2. gửi yêu cầu');
  setTimeout(() => {
    callback({ id: 'D01', status: 'online' });
  }, 1000);
}

getDevice((device) => {
  console.log('4. nhận dữ liệu:', device.status);
});

console.log('3. chương trình vẫn chạy');
```

Kết quả mong đợi

```
1. bắt đầu
2. gửi yêu cầu
3. chương trình vẫn chạy
4. nhận dữ liệu: online
```

Giải thích

setTimeout mô phỏng thời gian chờ server.

Callback chưa chạy ngay, mà được đưa vào hàng đợi.

Vì vậy dòng số 3 in ra trước dòng số 4.

18. Promise: thay callback bằng kết quả trong tương lai

Promise giúp tách rõ nhánh thành công và nhánh lỗi.

Ba trạng thái cần nhớ

pending: đang chờ kết quả.

fulfilled: hoàn tất thành công → then().

rejected: thất bại → catch().

Quy tắc đọc code Promise

resolve(value) chuyển dữ liệu sang then(value).

reject(error) hoặc throw chuyển lỗi sang catch(error).

finally() chạy sau cùng, dù thành công hay thất bại.

Ví dụ Promise

```
function getDevice(ok) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (ok) resolve({ id: 'D01', temp: 30 });
      else reject(new Error('Network error'));
    }, 1000);
  });
}

getDevice(true)
  .then(data => console.log('Temp:', data.temp))
  .catch(err => console.log('Error:', err.message))
  .finally(() => console.log('Done'));
```

19. Promise: bài tập đổi true/false

Thay true thành false để thấy catch() xử lý lỗi như thế nào.

Code tự kiểm tra

```
function login(username, password) {
  return new Promise((resolve, reject) => {
    if (username === 'ctu' && password === '123') {
      resolve({ token: 'ABC', role: 'student' });
    } else {
      reject(new Error('Sai tài khoản hoặc mật khẩu'));
    }
  });
}

login('ctu', '123')
  .then(user => console.log('Token:', user.token))
  .catch(err => console.log('Login fail:', err.message));
```

Kết quả mong đợi

```
Token: ABC

Nếu gọi login('ctu', '999'):
Login fail: Sai tài khoản hoặc mật khẩu
```

Câu hỏi cho sinh viên

Dữ liệu token đi qua resolve hay reject?

Lỗi sai mật khẩu đi vào then() hay catch()?

Vì sao Promise giúp code dễ đọc hơn callback lồng nhau?

20. async/await: viết Promise như các bước tuần tự

async/await không thay thế Promise; nó là cách viết Promise dễ đọc hơn.

Cách đọc

async đặt trước function: hàm luôn trả về Promise.

await chờ Promise hoàn tất rồi lấy kết quả.

try/catch dùng để bắt lỗi từ await.

Trong khi await, các tác vụ khác của chương trình vẫn có thể tiếp tục.

Khi nào dùng?

Gọi API, đọc file, truy vấn CSDL, hoặc các tác vụ cần chờ kết quả.

Viết lại Promise bằng async/await

```
async function main() {
  try {
    const device = await getDevice(true);
    console.log('Temp:', device.temp);
  } catch (err) {
    console.log('Error:', err.message);
  }
}

main();
```

Kết quả mong đợi

```
Temp: 30
```

Lưu ý: getDevice(true) là hàm Promise ở slide trước.

21. async/await: ví dụ gọi API có kiểm tra lỗi

Đây là mẫu code nên dùng trong các bài lập trình mạng.

Mẫu code API

```
async function loadDevice() {
  try {
    const res = await fetch('/api/device/D01');

    if (!res.ok) {
      throw new Error('HTTP ' + res.status);
    }

    const data = await res.json();
    const temp = data.device?.sensor?.temp ?? 'no data';

    console.log('Temperature:', temp);
  } catch (err) {
    console.error('Cannot load:', err.message);
  }
}

loadDevice();
```

Giải thích từng bước

await fetch(...): chờ server trả response.

res.ok: kiểm tra HTTP 200–299. Nếu lỗi thì throw.

await res.json(): chờ chuyển body thành object JS.

?. và ?? giúp đọc JSON an toàn khi thiếu trường.

catch bắt cả lỗi mạng, lỗi HTTP và lỗi parse JSON.

Bài tập: mock API trả thiếu sensor để kiểm tra “no data”.

22. Cùng một bài toán: 3 cách viết

Bài toán: lấy nhiệt độ thiết bị rồi in ra console.

Callback

Phù hợp với sự kiện đơn giản.

Khi nhiều bước phụ thuộc nhau, code dễ lồng sâu.

Cần quy ước rõ callback(err, data).

```
readTemp((err, temp) => {
  if (err) return log(err);
  console.log(temp);
});
```

Promise

Tách rõ then và catch.

Dễ nối chuỗi nhiều bước hơn callback.

Dùng Promise.all cho tác vụ song song.

```
readTemp()
  .then(temp => console.log(temp))
  .catch(err => log(err));
```

async/await

Dễ đọc như thuật toán tuần tự.

Phổ biến khi viết API client/server.

Kết hợp try/catch để xử lý lỗi.

```
try {
  const temp = await readTemp();
  console.log(temp);
} catch (err) {
  log(err);
}
```

Gợi ý cho sinh viên: học hiểu callback trước, sau đó dùng Promise và async/await để viết code mạng rõ ràng hơn.

23. Bài tập kiểm tra tổng hợp

Mục tiêu: sinh viên tự kiểm tra cả callback, Promise và async/await.

Code tự chạy

```
function mockApi(ok) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (ok) resolve({ device: { sensor: { temp: 31 } } });
      else reject(new Error('Mất kết nối server'));
    }, 500);
  });
}

async function test() {
  try {
    const data = await mockApi(true);
    console.log(data.device?.sensor?.temp ?? 'no data');
  } catch (err) {
    console.log(err.message);
  }
}

test();
```

Kết quả mong đợi

```
31

Nếu gọi mockApi(false):
Mất kết nối server

Nếu resolve({ device: {} }):
no data
```

Yêu cầu nộp bài

Giải thích vì sao cần await.

Chỉ ra dòng nào xử lý lỗi.

Chỉ ra dòng nào xử lý dữ liệu thiếu trường.

Đổi mockApi(true/false) và ghi lại kết quả.

CÁM ƠN

Câu hỏi & thảo luận